

Code Review — victor-vectori (module 1–5, exercițiile 1–5)

Review organizat pe cele 5 module ale fișei recapitulative, câte primele ~5 exerciții din fiecare. Codul NU a fost modificat — totul e doar aici.

Progres față de review-ul anterior. Am reverificat, linie cu linie, criticele din runda trecută. B1 (operatorul virgulă la cmmmc), B2 (comparație de parități înlănțuită), B3 ($v[i-1]$ la $i=0$), B5/B6 (dublete $i < d$), B7 (sortezi aux, afișezi v), B8 ($d=8$ pe 6 elemente) — **toate sunt rezolvate în codul actual**. Fișierele SOLUTII_REVIEW.pdf și versiunea veche a acestui review descriau acele bug-uri; acum sunt istorie. Ce urmează e ce a mai rămas + ce e nou.

■ Critice

B1 — progresieAritmetica nu verifică prima pereche de cifre (cifra cea mai semnificativă)

- realizareaUnuiNouVector.h:95 — while ($n > 9$) (folosită de solutie1c, realizareaUnuiNouVector.h:111)

Bucula se oprește când n a ajuns la **o singură cifră** — deci ultima diferență, dintre cele mai semnificative două cifre, nu se compară niciodată.

Urmărește pe 9420 cu rația 2 (ar trebui respins, $9 - 4 = 5 \neq 2$):

```
cifu=0, n=942
n=942>9 : cifp=2, 2-0=2 ok, cifu=2, n=94
n=94 >9 : cifp=4, 4-2=2 ok, cifu=4, n=9
n=9 >9 ? nu → stop → return true // dar 9-4=5 n-a fost verificat!
```

9420 trece testul deși nu e progresie. **Ce înveți:** condiția de oprire a buclei „pe cifre” e aproape mereu while ($n > 0$) (sau $n != 0$), nu $n > 9$. Cu $n > 9$ sari fix ultima verificare. Fix: while ($n > 0$) — atunci la ultimul pas cifp ia cifra cea mai semnificativă și diferența ei se compară înainte ca n să devină 0.

B2 — cmmdc prin scăderi intră în buclă infinită dacă unul din numere e 0

- tema2.h:680–690 și corectareTema2.h:248–260 — while ($a != b$) { if ($a > b$) $a -= b$; if ($b > a$) $b -= a$; }

Pe cmmdc(45, 0): $a=45$, $b=0$, $a > b \rightarrow a=45-0=45$, $b > a$ fals, la infinit — a și b nu se ating niciodată. E latent (azi e chemat doar prin cmmmc, care are gardă pe 0), dar bomba stă amorsată: orice apel viitor cu un 0 blochează programul.

Ai deja varianta sigură (Euclid cu rest) în chiar aceste fișiere — dublete.h:18, realizareaUnuiNouVector.h:59: while ($b != 0$) { int $r = a \% b$; $a = b$; $b = r$; }. Aceea se oprește natural pe 0. Folosește peste tot aceeași variantă, nu două cmmdc diferite în proiect.

■ Importante

M1 — minim / maxim mută vectorul și pornesc de la o valoare greșită

- `realizareaUnuiNouVector.h:500–512 (minim), :514–526 (maxim) — int min = v[0]; apoi v[i] = abs(v[i]);`

Două probleme suprapuse în `solutie3a (realizareaUnuiNouVector.h:528)`:

1. **Efect secundar:** `v[i] = abs(v[i])` rescrie vectorul primit. După `minim(v, d)`, vectorul original e deja modificat — `maxim(v, d)` nu mai vede datele reale. O funcție care se numește „minim” nu trebuie să schimbe ce primește.
2. **Inițializare greșită:** `min = v[0]` ia valoarea *înainte* de `abs`. Pe `{-15, -5, 0, 35, 45, 25}`, `min` rămâne `-15` (nu se mai actualizează, pentru că toate valorile `abs` sunt $\geq 0 > -15$), deci `minim` întoarce `-15`, nu `0`.

Corect: fie lucrezi pe `abs` local (`int x = abs(v[i]);`) fără să atingi `v`, fie pornești `min/max` din prima valoare *deja* prelucrată și compari de la `i = 1`.

M2 — aceleași funcții definite în mai multe header: bombă de compilare

prim apare în 6 fișiere (`exercitiiVectori.h:71`, `tema2.h:8`, `realizareaUnuiNouVector.h:186`, `dublete.h:6`, `corectareTema2.h:9`, `sortareaVectorilor.h:84`); afisare, `cmmdc`, `palindrom`, `cifControl` la fel, în mai multe.

Azi merge fiindcă `vectori.cpp:2` include un singur header. În clipa în care incluzi două (`dublete.h + tema2.h`), compilatorul dă `function redefinition`. **#pragma once NU te apără** — el previne doar includerea *aceluiași* fișier de două ori, nu coliziunea de nume *între* fișiere. Soluție la nivelul tău: un singur `utile.h` cu funcțiile comune, iar headerele de teme doar îl includ.

M3 — baza2 promite `long long` dar acumulează în `int`

- `tema2.h:230–241 — long long baza2(int n) cu int intern? Nu — aici tipurile interne sunt corecte (long long b2, long long p). Corect.`

(Verificat: în versiunea curentă variabilele de lucru sunt `long long`. Punctul din review-ul vechi nu se mai aplică — îl las marcat ca rezolvat.)

■ Cleanups

- **C1 — bubble sort rescris de ~8 ori** în `tema3.h` (`sortatPrinBubbleSortDescrescator:8`, `sortatCrescatorPartial:182`, `BubbleSortCrescator:356`, `sortatCrescatorSuma:460`, `sortatCrescator:494...`). Diferă doar *criteriul de comparație*. Până la o funcție cu criteriu parametru, măcar una crescător + una descrescător, refolosite.
- **C2 — `if (cond) return true; return false;` → `return cond;`** (`realizareaUnuiNouVector.h:141`, `corectareTema2.h:31`, `dublete.h` și altele).
- **C3 — `bool` cu 0/1:** `bool vf = 1 (tema2.h:176)`, `while (sortat == false) → bool vf = true, while (!sortat)`.
- **C4 — `;; dublu`** la `exercitiiVectori.h:50`.
- **C5 — cod mort:** `baza2 / ctCifre1` rămase la finalul `MunteVale.h:140–163` fără apel; `int poz = i + 1` nefolosit (`tema2.h:670`, `corectareTema2.h:234`).

- **C6 – patratPerfect cu float:** `sqrt(n) == (int)sqrt(n)` (tema3.h:278, corectareTema2.h:66) e fragil pe valori mari. Ai varianta bună `(rad * rad == n)` în tema4.h:167 și realizareaUnuiNouVector.h — folosește-o peste tot.

Ce e corect pe module (ex. 1–5) — confirmări

Ca să știi ce e solid, nu doar ce e de reparat:

Modul	Fișier	Exerciții verificate	Stare
1 — parcurgere/filtrare	exercitiiVectori.h	solutie1–solutie4	corecte
1 — vector nou	realizareaUnuiNouVector.h	solutie1a, 1b, 1d, 1e corecte; 1c are B1	1 bug
2 — cifre pe vector	tema2.h	solutie1a– solutie1e	corecte
3 — dublete	dublete.h	solutie1a– solutie1d	corecte (B5/B6 rezolvate)
4 — sortări	tema3.h, sorta- reaVectorilor.h	primele 5	corecte (B7/B8 rezolvate)
5 — ștergere/inserare	tema4.h	solutie1a– solutie1e	corecte

Before / After — criticele curente

#	Before (actual)	After (corect)
B1	<code>while (n > 9) { ... } în progresieAritmetica</code>	<code>while (n > 0) { ... }</code>
B2	<code>while (a != b) { if (a>b) a-=b; if (b>a) b-=a; }</code>	<code>while (b != 0) { int r = a % b; a = b; b = r; }</code>

Q&A — verifică-ți înțelegerea

1. În `progresieAritmetica`, pentru 9420 cu rația 2, câte perechi de cifre se compară efectiv și care pereche NU ajunge niciodată verificată? De ce `while (n > 0)` rezolvă, iar `while (n > 9)` nu?
2. `minim` din `realizareaUnuiNouVector.h` schimbă vectorul primit. Rulează `solutie3a` pe hârtie și spune ce valoare vede maxim la `v[0]` — și de ce `min` iese -15 în loc de 0.
3. De ce `#pragma once` NU previne eroarea de `function redefinition` când incluzi `dublete.h` și `tema2.h` în același `.cpp`? Ce anume previne el, de fapt?